

Spring Framework & Core Concepts

1. What is Spring?

Spring is a very popular **Java framework** that helps developers build large applications easily. It takes care of all the complicated "plumbing" (setup and infrastructure) so you can focus only on writing the code for your business rules. It makes applications organized, easy to test, and flexible.

2. Why do developers prefer using Spring instead of writing everything with plain Java?

Plain Java requires you to manually manage every object, connection, and configuration file, which is time-consuming and prone to errors. Spring does this management automatically using its **container**. It allows developers to write code that is loosely coupled, meaning components rely on contracts (interfaces) instead of specific classes, making the code much easier to change and test.

3. Can you break down the key building blocks or modules that make up the overall Spring ecosystem?

The core of the ecosystem is the **Spring Core** container, which handles dependency injection. Above that, key building blocks include:

- **Spring Data:** For database access (JPA, JDBC).
- **Spring Web/MVC:** For building web applications and REST APIs.
- **Spring Security:** For authentication and authorization.

4. What is the core difference between using Spring Boot and using the traditional Spring Framework setup?

The traditional Spring Framework requires extensive manual configuration, often using XML or many Java files. **Spring Boot** is an extension that bundles all that configuration automatically and provides sensible defaults. It lets you run your app immediately with minimal setup and includes an **embedded server** (like Tomcat).

IoC, DI, and Beans

5. In Spring's world, what do we mean when we say something is a "bean"?

A **bean** is simply a Java object that is created, assembled, and managed entirely by the Spring IoC Container. Instead of you writing new MyClass(), you tell Spring, "Here is this class, please manage it for me as a bean," and Spring handles its entire life cycle.

6. How would you explain IoC (Inversion of Control) and DI (Dependency Injection) without using heavy terminology?

Inversion of Control (IoC) means that the framework takes control of your application's object management. Instead of you creating objects, the Spring Container creates them for you. **Dependency Injection (DI)** is the way Spring achieves this; if your object needs another object to work, Spring simply provides (injects) it from the outside.

7. In what way does the IoC container take control of creating and wiring the objects in an application?

The IoC container reads your configuration (annotations) to find out which classes should be beans. It then instantiates those classes and stores them. If one bean needs another bean, the container automatically "wires" (connects) them together based on your instructions (like the `@Autowired` annotation), making the objects ready to use.

8. What different container implementations does Spring provide for managing objects?

The main implementations are the **BeanFactory** and the **ApplicationContext**. The **ApplicationContext** is the advanced version built on top of the **BeanFactory**. The **ApplicationContext** is almost always used in modern development because it adds features like event handling, internationalization, and component scanning.

9. Why do we use annotations like `@Configuration` and `@Bean`, and what do they help Spring understand?

We use these annotations to provide Spring with instructions about how to manage objects. **@Configuration** tells Spring, "This class defines bean configuration." **@Bean** is placed on methods inside a **@Configuration** class and tells Spring, "This method produces an object that should be managed as a bean."

10. What different scopes can a Spring bean have while running an application?

The two most common scopes are **singleton** and **prototype**. Other scopes include request, session, and application (used in web environments).

11. If you don't specify any scope, which scope does Spring automatically assign to the bean?

Spring automatically assigns the **singleton** scope by default. This means that the Spring Container will create only one instance of that bean, and every other part of the application will share that same single object.

12. Which design patterns form the backbone of Spring's internal architecture?

The backbone is built on the **Factory Pattern** (to create objects), the **Strategy Pattern** (to allow core components to be swapped), and the **Proxy Pattern** (used for AOP, security, and transaction management). The overarching principle is the **Dependency Inversion Principle (DIP)**.

13. When you use `@Autowired`, what kind of dependency injection does Spring normally choose by default?

When you use `@Autowired`, Spring prefers **Constructor Injection** if there is only one constructor. If you place `@Autowired` on a field or setter method, Spring uses **Setter Injection** or **Field Injection**. However, Constructor Injection is generally recommended as best practice because it makes the object's dependencies explicit.

14. How is `@Service` different from `@Component` in terms of intention and meaning?

Technically, `@Service` is just a specialized form of `@Component`. They both tell Spring, "Treat this class as a bean." However, `@Service` is used to clearly mark a class that holds **business logic** (e.g., calculating taxes). This improves code readability and separation of concerns.

15. What are the practical differences between injecting dependencies through constructors versus using setter methods?

Constructor Injection is better for mandatory dependencies because the object cannot be created unless all dependencies are provided. This guarantees immutability and thread safety. **Setter Injection** is better for optional dependencies that can be changed after the object is created, but it does not guarantee the dependency will ever be set.

16. What role does `@ComponentScan` play, and how does it allow Spring to find and register components automatically?

`@ComponentScan` tells Spring which packages to scan for classes annotated with component markers (like `@Component`, `@Service`, etc.). Spring reads this instruction and automatically finds those classes, registers them as beans, and prepares them for use. This saves you from having to manually list every single class you want Spring to manage.

17. In a Spring Boot project, what purpose do `application.properties` or `application.yml` files serve?

These files are the central location for configuring your application. They allow you to set simple values, like the server port (`server.port=8080`), database connection details, or logging levels. This lets you change these settings easily without having to modify and recompile the actual Java code.